INFO0027: File Deduplication Service

Final Architecture & Design Report

Group Members:
Franck Duval HEUBA
Mawa

Deadline: 2026-05-15

University of Liège

Declaration on Generative AI

During the preparation of this work, the authors used Gemini 1.5 Pro to: Brainstorm software architecture design, format the JSON Data Transfer Objects, provide guidance on Project Panama (FFM API) for native C interoperability, and structure this LaTeX report. After using these tools/services, the authors reviewed, heavily refactored, and edited the content to fit the specific constraints of the VirtualFileSystem environment and take full responsibility for the publication's content.

Contents

1 Architecture and Main Components

The deduplication service was designed with high cohesion and loose coupling to accommodate the diverging needs of the Frontend, Storage, and Photography teams. The core architecture relies on an abstraction layer separating the domain logic from the specific algorithms.

The system is bootstrapped via the `MyDeduplicationBootstrap` class, which acts as a Composition Root, instantiating and linking the following core components:

- **FrontendGate (`MyFrontendGate`):** Acts as the API facade for frontend JSON requests. It safely parses incoming inputs using DTOs and delegates the processing to the underlying engines, returning lazy streams to maintain low memory footprints.
- **StorageChecker (`MyStorageChecker`):** Intercepts file uploads on-the-fly. It optimizes the $O(N)$ comparison complexity down to $O(1)$ by using a size-based heuristic index (`VfsIndexCache`) before delegating deep content checks to the exact match engine.
- **EngineRouter:** A central dispatcher that analyzes incoming requests (e.g., "exact" vs "similar") and provides the appropriate deduplication strategy while managing the "Seamless Swap" mechanism.

Note: Please refer to the Annex for the complete UML Class Diagram.

2 Software Patterns

The solution heavily leverages established design patterns to ensure maintainability and flexibility without introducing unnecessary complexity.

2.1 Strategy Pattern

Rationale: The system requires three interchangeable deduplication engines (Pure Java, Native C, and OpenCV Similarity).
Implementation: All engines implement the `DeduplicationEngine` interface. This completely decouples the `FrontendGate` and `StorageChecker` from the algorithmic complexity.
Trade-offs: Introduces slightly more boilerplate code (interfaces and multiple classes) but guarantees that modifying the C-interop logic will never break the OpenCV logic.

2.2 Factory / Fallback Pattern

Rationale: The "Seamless Swap" requirement dictates that if the highly optimized C library fails to load (due to OS incompatibility or missing `.so` files), the system must fallback to Pure Java.
Implementation: The `EngineRouter` acts as a Factory. It captures `UnsatisfiedLinkError` during the instantiation of the Native C engine and transparently returns the `PureJavaHashEngine` instead.

2.3 Data Transfer Object (DTO)

Rationale: The frontend communicates strictly via JSON encoded strings.
Implementation: Java `Records` (`DedupRequestDto`, `DedupResponseDto`) were used to model the JSON structure. Records provide inherent immutability and thread-safety, making them ideal for the asynchronous streams required by the `acceptStream` method.

3 Quality Assurance and Test Plan

Quality assurance was driven by the ISO/IEC 25010:2024 Product Quality Model. We focused on three main characteristics critical to a core infrastructure service:

Main Characteristic	Sub-characteristic	Application & Tracking
Performance Efficiency	Time Behavior	Tracked via the `VfsIndexCache`. By filtering files by exact byte size before cryptographic hashing, we drastically reduced CPU cycles during storage checks.
Reliability	Fault Tolerance	The "Seamless Swap" mechanism ensures the application remains fully operational (falling back to Java SHA-256) even if native FFM API calls fail.
Maintainability	Modularity	Assessed through strict adherence to the VirtualFileSystem (VFS) interface. No hardcoded `java.io.File` operations exist in the core engines, allowing future migration to cloud storage.

Table 1: ISO/IEC 25010 Quality Characteristics applied to the project.

Test Plan: Testing was divided into two phases: 1. *Unit Testing:* Utilizing JUnit 5 to mock the `VirtualFileSystem` and verify the routing logic of the `EngineRouter` and the JSON serialization in the `FrontendGate`. 2. *Integration Testing:* Generating the standalone `Deduplicator.jar` via Gradle's Shadow plugin and running it against the provided testing script to validate the `ServiceLoader` discovery mechanism.

4 Design Process and Challenges

The most significant architectural challenge was integrating the legacy C library securely into the modern Java ecosystem. Initially, JNI (Java Native Interface) was considered, but we adopted Project Panama (FFM API) using `jextract`.

Challenge overcome: The C function `FDDump` returns a `char**` (array of strings) with no explicit length, separated by NULL pointers. Reading this safely in Java without causing a JVM segmentation fault required careful manipulation of the `MemorySegment` API to reinterpret the memory layout size dynamically.

Furthermore, implementing the OpenCV similarity engine required adapting standard `imread` operations to work strictly with the `VirtualFileSystem`. We overcame this by reading VFS byte arrays into memory and utilizing `Imgcodecs.imdecode` to bypass physical disk access.

5 Limitations and Future Improvements

While robust, the current design has a few recognized limitations:

- **Native Memory Leaks:** The provided C library lacks an `FDFree()` function. Consequently, the memory allocated by the native code during `FDInit()` and `FDCheck()` cannot be explicitly freed from the Java `Arena`, potentially leading to memory bloat over prolonged execution periods.
- **In-Memory Cache Scalability:** The `VfsIndexCache` groups all files by size in a `ConcurrentHashMap`. While extremely fast, if the VFS grows to billions of files, this metadata map might exceed

JVM heap limits. A future iteration should offload this index to an external key-value store like Redis.